Proto Design Document

Phillip Colella
Daniel T. Graves
Christopher L. Gebhart
Brian Van Straalen

April 10, 2026

1 Introduction

`Proto` is a lightweight library designed for efficient solution of differential equations on domains composed of unions of structured, logically rectangular grids. The goal of `Proto` is to provide a fluent, intuitive interface that separates the complexities of *designing* and *scheduling* an algorithm. As both the tools and applications of high performance computing become continuously more intricate, libraries such as `Proto` will become more and more necessary to efficiently solve problems in a timely manner.

In general, variables which refer to tokens in `Proto` are written using a `monospaced font`. Vector-like objects are written in lower-case ***bold***, number types use black-board font ($\mathbb{R}$), and sets are written in math calligraphy font ($\mathcal{S}$). Occasionally words are typed in bold-face simply for **emphasis**.

Variable Name	Variable Definition
$\boldsymbol{D}$ or `DIM`	Number of space dimensions
$\mathbb{Z}^{\boldsymbol{D}}$	$\boldsymbol{D}$-dimensional Space of Integers
$\boldsymbol{h}, \boldsymbol{i}, \boldsymbol{j}, \boldsymbol{k}, \boldsymbol{l}, \boldsymbol{r}, \boldsymbol{s}, \boldsymbol{t} \ldots$	Points in $\mathbb{Z}^{\boldsymbol{D}}$
$\boldsymbol{B}$	A Box, i.e., Rectangular subset of $\mathbb{Z}^{\boldsymbol{D}}$
$\mathrm{Box}(V)$	Domain of multidimensional rectangular array V
$[\boldsymbol{l}, \boldsymbol{h}]$	Box with low and high corners ($\boldsymbol{l}$, $\boldsymbol{h}$
$\mathbb{T}$	Space of numbers of type `T`
$\boldsymbol{e}^d$	Unit vectors in direction d
$\boldsymbol{u}$	Point in $\mathbb{Z}^{\boldsymbol{D}}$ all of whose entries are 1

2 Spatial Discretizations on Rectangular Grids

2.1 Primitive Data Types, Points, Boxes, and Data Arrays

These are the primitive objects used to define discretizations on structured grids. This is embedded in the full C++ language, so that on CPUs you have access to the full C++ type system. However, on GPUs, one typically does not have access to the complete C++, since many of the constructs in C++ either are not available (e.g. STL), or cause a catastrophic loss of performance (malloc). So `Proto` uses a restricted set of types to implement structured grid calculations at high performance.

- `Array<T,N>` is a fixed-size array type similar to `std::array`, but meant to be used only with primitive data types (`T = float,double,int` ...) and `Arrays` of such objects, e.g. `Array<Array<int,DIM>,DIM>`. These can be passed as arguments into pointwise functions.
- $\mathbb{Z}^{\boldsymbol{D}}$, the $\boldsymbol{D}$-dimensional rectangular lattice of points are used to represent discretizations of space: $\boldsymbol{j} \in \mathbb{Z}^{\boldsymbol{D}}$ represents a point in physical space $jh \in \mathbb{R}^{\boldsymbol{D}}$, where h is the mesh spacing.
- A Box $\boldsymbol{B}$ is a rectangular subset of $\mathbb{Z}^{\boldsymbol{D}}$. $\boldsymbol{B} = [\boldsymbol{l}, \boldsymbol{h}]$ where $\boldsymbol{l}, \boldsymbol{h} \in \mathbb{Z}^{\boldsymbol{D}}$, $\boldsymbol{l} \leq \boldsymbol{h}$ are the low and high corners of $\boldsymbol{B}$.
- Data arrays are maps from Boxes to scalar, vector, or two- and three-tensors.

$$\phi : B \to \mathbb{T}^C \times \mathbb{T}^D \times \mathbb{T}^E \ , \ C, D, E \geq 1, \tag{1}$$

  where $\boldsymbol{B}$ is a Box, and $\mathbb{T}$ is one of the real numbers ($\mathbb{R}$), the complex numbers ($\mathbb{C}$), or the integers ($\mathbb{Z}$). Data arrays are meant to approximate a field defined on a subset of $\mathbb{R}^{\boldsymbol{D}}$.

$$\phi_{\boldsymbol{j}} \approx \Phi(\boldsymbol{j}h) \ , \ \Phi : \Omega \subseteq \mathbb{R}^{\boldsymbol{D}} \to \mathbb{T}^C \times \mathbb{T}^D \times \mathbb{T}^E.$$

2.2 Stencils and Pointwise Operators

A broad range of discretizations PDE operators can be represented as the composition of **stencil** and **pointwise** operators applied to data arrays.

2.2.1 Stencils

Stencil operators take as input a data array, and produce a new data array computed as the sum of shifts of the original data array, each multiplied by a coefficient that depends only on the shift.

$$\psi := S(\phi) \ \textbf{on} \ \boldsymbol{B}' \ , \ S(\phi)_{\boldsymbol{i}} = \sum_{\boldsymbol{s} \in \mathcal{S}} a_{\boldsymbol{s}} \phi_{\boldsymbol{i}+\boldsymbol{s}},$$

where the coefficients $a_{\boldsymbol{s}} \in \mathbb{T}$, $\boldsymbol{\mathcal{S}}$ is a finite subset of $\mathbb{Z}^{\boldsymbol{D}}$, and $\boldsymbol{B}' \subseteq \cap_{\boldsymbol{s}\in\mathcal{S}}(\boldsymbol{B}(\phi) - \boldsymbol{s})$. Such operators appear as finite difference approximations to constant-coefficient differential operators. For multilevel algorithms such as AMR and multigird, we will want more general versions of stencil operators corresponding to strided access through the data array. A more general form of such a stencil is given by

$$\begin{aligned} \psi &:= S(\phi) \textbf{ on } \boldsymbol{B}' \\ S(\phi)_{\boldsymbol{r}^d\boldsymbol{i}+\boldsymbol{t}} &= \sum_{\boldsymbol{s}\in\mathcal{S}} a_{\boldsymbol{s}}\phi_{\boldsymbol{r}^s\boldsymbol{i}+\boldsymbol{s}} \ , \ \boldsymbol{i} \in \boldsymbol{B}' \end{aligned} \tag{2}$$

where $\boldsymbol{ri} = (r_0 i_0, \ldots, r_{\boldsymbol{D}-1} i_{\boldsymbol{D}-1}) \in \mathbb{Z}^{\boldsymbol{D}}$. Note that, if $\boldsymbol{r}^d \neq \boldsymbol{u}$, the application of the stencil assigns values to a strided subset of the data array on the left-hand side. While the action of a stencil on a data array depends on the inputs, stencils themselves, as specified by the collection of offsets and weights $\{(\boldsymbol{s}, a_{\boldsymbol{s}}) : \boldsymbol{s} \in \mathcal{S}\}$ and the input and output stride and output offset $(\boldsymbol{r}^s, \boldsymbol{r}^d, \boldsymbol{t})$ have a mathematical meaning independent of the inputs. In particular, if $\boldsymbol{r}^s, \boldsymbol{r}^d = \boldsymbol{1}$, and $\boldsymbol{t} = \boldsymbol{0}$, stencils can be added, multiplied by scalars, and composed to obtain new stencils.

$$S = tS_1 + S_2 \Leftrightarrow S(\phi)_{\boldsymbol{i}} = \sum_{\boldsymbol{s}\in\mathcal{S}_1\cup\mathcal{S}_2} (ta^1_{\boldsymbol{s}} + a^2_{\boldsymbol{s}})\phi_{\boldsymbol{i}+\boldsymbol{s}} \tag{3}$$

$$S = S_1 \cdot S_2 \Leftrightarrow S(\phi)_{\boldsymbol{i}} = \sum_{\boldsymbol{s}\in\mathcal{S}}, (\sum_{\boldsymbol{s}_1,\boldsymbol{s}_2:\boldsymbol{s}_1+\boldsymbol{s}_2=\boldsymbol{s}} a^1_{\boldsymbol{s}_1} a^2_{\boldsymbol{s}_2})\phi_{\boldsymbol{i}+\boldsymbol{s}} \ , \ \mathcal{S} = \{\boldsymbol{s}_1 + \boldsymbol{s}_2 : \boldsymbol{s}_1 \in \mathcal{S}_1, \boldsymbol{s}_2 \in \mathcal{S}_2\}. \tag{4}$$

Algebraic operations and composition enables the construction of complex stencils from simpler ones.

2.2.2 Pointwise Operators

Pointwise operations promote functions over the various primitive types $\mathbb{T}$ to be functions on the corresponding data array types. For example, if $f(x) = f : \mathbb{T}^M \to \mathbb{T}^N$, then, for any $\phi : \boldsymbol{B} \to \mathbb{T}^M$ the operation

$$f(\phi_{\boldsymbol{j}}) \ , \ \boldsymbol{j} \in \boldsymbol{B} \cap \boldsymbol{B}'$$

produces a data array taking on values in $\mathbb{T}^N$. Following [**?**], we denote the above operation on data arrays

$$f@(\phi) \textbf{ on } \boldsymbol{B}'.$$

This can be substantially generalized. If we have a function f of the form

$$f : \mathbb{T}_1^{M_1} \times \cdots \times \mathbb{T}_P^{M_P} \to \mathbb{T}_0,$$

we denote by

$$f@(\zeta^1, \ldots, \zeta^P) \textbf{ on } \boldsymbol{B}',$$
$$f@(\zeta^1, \ldots, \zeta^P)_{\boldsymbol{j}} = f(\zeta^1_{\boldsymbol{j}}, \ldots \zeta^P_{\boldsymbol{j}}) \ , \ \boldsymbol{j} \in \boldsymbol{B}_1 \cap \cdots \cap \boldsymbol{B}_P \cap \boldsymbol{B}'$$
$$\zeta_p : \boldsymbol{B}_p \to \mathbb{T}_p^{M_p} \ , \ p = 1, \ldots, P.$$

This same notation is used for tensor-valued arguments. Finally, we can replace any subset of the arguments with non-map values, i.e. for the same function given above:

$$f@(\zeta^1, \ldots, \zeta^{P-1}, \chi)_{\boldsymbol{j}} = f(\zeta^1_{\boldsymbol{j}}, \ldots \zeta^{P-1}_{\boldsymbol{j}}, \chi) \ , \ \boldsymbol{j} \in \boldsymbol{B}_1 \cap \cdots \cap \boldsymbol{B}_{P-1} \cap \boldsymbol{B}'$$
$$\zeta_p : \boldsymbol{B}_p \to \mathbb{T}_p^{M_p} \ , \ p = 1, \ldots, P-1 \ ; \ \chi \in \mathbb{T}_P^{M_P}.$$

3 `Proto` Representation of Structured Grid Operations

In `Proto`, We have C++ classes that correspond to points, boxes, and data arrays; and operators corresponding to Stencils and pointwise operations. We provide a summary of these classes below. The full documentation is contained in the Doxygen-annotated header files for `Proto`.

3.1 Points, Boxes, and Data Arrays

3.1.1 `Point`

A `Point` is a vector in $\mathbb{Z}^{\boldsymbol{D}}$. `Proto` uses `Points` to define locations in $\mathbb{Z}^{\boldsymbol{D}}$ as well as offsets of subsets of $\mathbb{Z}^{\boldsymbol{D}}$. The dimensionality of our space ($\boldsymbol{D}$) is set by setting the compile-time constant `DIM`.

- Construction. `Points` can be constructed from C arrays of length `DIM`, and from integer tuple literals `Point({1,2,1,2,1,2})`. In the latter case, the number of elements in the integer tuple can be greater than `DIM`, with the ones exceeding the length being ignored. There are also static member functions that return a `Point` whose components are all zeros (`Point::Zeros()`), all ones (`Point::Ones()`), and the unit vector in the dth direction (`Point::Basis(d)`, $d = 0, \ldots,$ `DIM` $- 1$.

- Arithmetic Operators. `Points` have addition, componentwise multiplication, multiplication and division by an integer (in the latter case, with rounding toward $-\infty$).

- Logical Operators. `p1 == p2`, `p1 != p2`. The inequality operators e.g. `p1 > p2` return true if they are true for each pair of components, i.e. the usual partial ordering on $\mathbb{Z}^{\boldsymbol{D}}$.

- Indexing Operators. `p[d]`, $d = 0, \ldots, \boldsymbol{D} - 1$. returns the dth component of `p`. `p[d]` can appear as the left-hand side in an assignment.

3.1.2 `Box`

A `Box` is a $\boldsymbol{D}$ dimensional rectangle in $\mathbb{Z}^{\boldsymbol{D}}$, defined by a pair of `Points` specifying the low and high corners of the Box.

- Constructors. `Box(low,high)`, where `low, high` are `Points`. If `(high >= low) == false`, then the `Box` is an empty `Box`.
- Accessors and Queries `B1 == B2`, `B1 != B2`, `contains(Point a_pt)`, `onBoundary(Point a_pt)`, `empty()` all return `bools`.
- Operators and Transformations. `B1 & B2` returns the intersection of the two `Boxes`. `coarsen`, `refine`, `shift` and other transformation operations compute `Boxes` needed to construct `BoxData` targets for various structured-grid algorithms.
- Iterators. `BoxIterator` iterates over the `Points` in a `Box`. It is accessible using the standard iterator syntax: `bx.begin()` returns a `BoxIterator` for the `Box bx` initialized at the starting location; `it.done()` returns a `bool` telling whether the iterator has reached to end; and `++it` increments the iterator.

3.1.3 BoxData

A `BoxData<T,C,D,E>` represents a data array defined on a `Box` domain of the form (1), where C, D, and E are positive integers which define the sizes of component indices 0, 1, and 2 respectively. Operations on `BoxData` include the following:

- Construction.

  `BoxData(const Box& a_bx)`, `define(const Box& a_bx)` define a `BoxData` over the `Box a_bx`. We can also control whether the data is allocated from the heap (`defineMalloc`) or from a specialized stack manager (`defineStack`) that provides a low-overhead way of allocating temporaries; the default is the latter.

  Copy construction and assignment are deleted. However, move construction and assignment is supported: `BoxData (BoxData< T, C, D, E > &&a_src)`, `BoxData& operator= (BoxData< T, C, D, E > &&a_src)`.

- Algebraic Operations. `operator{+,*,-,/}(const BoxData& a_bd)` returns a `BoxData` on the intersection of the `Boxes` on which the two inputs are defined. `operator{+=,*=,-=,/=}(const BoxData& a_bd)` computes the same set of values, but updates the left-hand side in place. We also define `operator(){+=,*=,-=,/=}()(T a_t)` to apply the operator to each value in the `BoxData`, and update the left-hand side in place. `setVal(T a_t)` sets all of the values of the `BoxData` to `a_t`. `min`, `max`, and `absMax`: Computes the min, max, or absolute max of a `BoxData`.

- Slicing and Shifting. Operations on BoxDatas are specific to the type; in particular, they cannot be performed on a BoxData with one set of component ranges to produce a BoxData with a different set of component ranges. For example,

```
BoxData<T,1,1,1> slice(const BoxData< T,C,D,E> & a_src,
                       unsigned int a_c,
                       unsigned int a_d = 0,
                       unsigned int a_e = 0)
```

  creates a scalar-valued BoxData that points to the slice corresponding to component (a_c,a_d,a_e). The resulting BoxData can then be used in functions taking scalar-valued BoxDatas. Similarly, the function

```
BoxData<T,C,D,E> alias(BoxData<T,C,D,E> &a_original,
                       const Point &a_shift)
```

  creates a BoxData with the same data pointer, but the Box offset by a_shift.

3.2 Pointwise Operations: forall, forallInPlace

The various versions of forall inplement the pointwise operations defined by $f@(\dots)$. We will illustrate the syntax of forall by a sequence of examples.

3.2.1 forall,forall_p

For the function $f(x, \alpha) := \alpha x^2$ we implement $f@(\phi, \alpha)$ **on** $\boldsymbol{B}$.

```
PROTO_KERNEL_START
void
f_F(
    Var<double>& a_output,
    const Var<double>& a_phi,
    double a_factor)
{
    a_output(0) = a_factor*a_phi(0)*a_phi(0);
}
PROTO_KERNEL_END(f, f_F);
```

We then call forall using f as one of the argument.

```
int main(argc, argv)
{
```

```
  Box bx = Box::UnitBox(100);
  Box bx0 = bx.grow(Point::Ones(-1));
  BoxData<double> phi(bx);
  // set phi to be a collection of random values.
  phi.random(0.,1.);
  BoxData<double> phisq = forall<double,1,HOST>(f,phi,2.0);
  auto phisq0 = forall(f,phi,4.0,bx0);
}
```

- The arguments of the pointwise function `f` distinguish between ones that access individual values of `BoxDatas` (`a_output`, `a_phi`), and primitive types (`a_alpha`).
- The first argument of `f` corresponds to the return `BoxData`.
- `forall` returns a `BoxData` using `C++ move` semantics, i.e. it computes the return `BoxData` internally, then moves it to the right-hand side.
- The first call to `forall` returns a `BoxData` defined on `bx`; the second one returns a `BoxData` defined on `bx0`. In the second case, the compiler is inferring from the code the template parameters for `forall` and the type of the return value. If the compiler is unable to do that, it will return an error and the template parameters / return type will have to be written out explicitly, as done in the first call.

The versions of `forall` with a suffix `_p` allow one to call pointwise functions with the points in a `Box` as one of the arguments.

- The above program creates a `BoxData phi`, that has been initialized to a smooth radially symmetric cosine bell.
- `h5.writePatch` writes `phi` to an HDF5 file named `phi100.hdf5`, which can be viewed using VisIt or ParaView (see `include/base/Proto_HDF5.H`).
- This is an example of how one can write dimension-independent code in `Proto`. The same code is runs for any spatial dimension `DIM` > 0, which is set at compile time.

3.2.2 `forallInPlace`, `forallInPlace_p`

`forallInPlace` is a version of `forall` that updates all of the non-const `BoxData` arguments in place. This might be used for a variety of circumstance:for example, when multiple arguments are being set. In all cases, the `BoxData` arguments are assumed to be completely defined; in particular, the `Boxes` are all defined, in contrast to `forall`, in which the `Box` over which the output is defined is set as part of the calculation.

```
include "Proto.H"
PROTO_KERNEL_START
template<typename T, MemType MEM>
void
init_F(
        Point a_pt,
        Var<T,1,MEM>& a_output,
        Array<T,DIM,MEM> a_center,
        T a_h)
{
  T radius = 0.;
  for (int d = 0; d < DIM; d++)
    {
      T location += (a_pt[d] + .5)*a_h - a_center[d];
      radius += location*location;
    }
    radius = sqrt(radius);
    if (radius  < .25)
    {
      output(0) = pow(cos(radius*M_PI/.5),6);
    }
    else
    {
      output(0) = 0.;
    }
}
PROTO_KERNEL_END(init, init_F);
```

Figure 1: Pointwise function to initialize cosine bell.

```
int main(argc, argv)
{
  N = 100;
  Box bx = Box::UnitBox(N);
  double h = 1.0/N;
  Array<double,DIM> center={.5,.5,.5};
  BoxData<double,1,HOST> phi = forall<double,1,HOST>(init1,bxcenter);
  HDF5Handler h5;
  h5.writePatch(h,phi,"phi"+to_string(N));
}
```

```
include "Proto.H"
PROTO_KERNEL_START
template<typename T, MemType MEM>
void
momentumToVelocity_F(
              Var<T,DIM+2,MEM>& a_flowVars
              T a_gamma)
{
  T rho = a_flowVars(0);
  T energy = flowVars(DIM+1);
  T ke = 0;
  for (int d = 0; d < DIM; d++)
  {
    T vel = flowVars(d+1)/rho;
    ke += vel*vel;
    a_flowVars(d+1) = vel;
  }
  ke *= .5*rho;
  p = (energy - ke)*(gamma - 1.0);
  a_flowVars(DIM+1) = p;
}
PROTO_KERNEL_END(eos, eos_F);
template<typename T, MemType MEM>
void
intensive_F(
            Var<T,DIM+2,MEM>& a_flowVars
            )
{
  T rho = a_flowVars(0);
  T energy = flowVars(DIM+1);
  T ke = 0;
  for (int d = 0; d < DIM; d++)
  {
    T vel = flowVars(d+1)/rho;
    ke += vel*vel;
    a_flowVars(d+1) = vel;
  }
  ke *= .5*rho;
  a_flowVars(DIM+1) -= ke;
}
PROTO_KERNEL_END(template<typename T, MemType MEM>
void eos_F(
```

```
          Var<T,DIM+2,MEM>& a_flowVars
         T a_gamma)
{
  flowVars(DIM+1) *= (gamma - 1.0);
}
PROTO_KERNEL_END(eos, eos_F); , eos_F);
int main(argc, argv)
{
  N = 100;
  Box bx = Box::UnitBox(N);
  double gamma = 5.0/3.0;
  BoxData<double,DIM+2,HOST> U;
  // ... initialize conserved quantities U (omitted).
  BoxData<T,DIM+2,HOST> W = forall(intensive,U);
  forallInPlace(eos,W,gamma);
}
```

3.3 Stencils

`Stencil`s are created using `operator*(T coef,Shift shift)` which produces an atomic `Stencil` object with one coefficient `coef` and associated offset `shift`. These atomic `Stencil`s are added together to create the final `Stencil`. An example of this process is shown in Listing ??

```
// 2nd Order Second Derivative along axis 0:
Stencil<double> D2 = 1.0*Shift::Basis(0)
                   + 1.0*Shift::Basis(0,-1)
                   + (-2.0)*Shift::Zeros();

// 2nd Order Laplacian:
Stencil<double> L = (-2.0*DIM)*Shift::Zeros();
for (int dir = 0; dir < DIM;dir++)
{
  L += 1.0*Shift(Point::Basis(dir))
      +1.0*Shift(-Point::Basis(dir));
}
```

4 Data On Unions of Rectangles

Initially, `Proto` was designed to provide a replacement to callbacks to `Fortran` in `Chombo` as a mechanism for getting high-performance single-patch calculations that could be ported to

GPUs and other accelerator-based systems. However, the natural granularity for specifying whether the data is resident on the host or device is that of a union of rectangles, consistent with the idea that `Chombo` runs on the host and `Proto` can run on on either the device or host. For that reason, we implement a specialized version of the `Chombo` functionality for representing data on unions of rectangles.

Disjoint unions of fixed-sized boxes, and their assignment to processors.

$$\Omega = \bigcup_k \Omega_k \subset B_{PD} \tag{5}$$

$$\mathcal{R}_{\boldsymbol{r}}(\mathcal{C}_{\boldsymbol{r}}(B_{PD})) = B_{PD} \ , \ \Omega_k = [\boldsymbol{r}\boldsymbol{i}_k, \boldsymbol{r}(\boldsymbol{i}_k + \boldsymbol{u}) - \boldsymbol{u}]$$

For each Ω_k we assign an MPI rank p_k.

This is represented in `Proto` using `Proto::DisjointBoxLayout`, which implements a variation of the `Chombo::DisjointBoxLayout` API specialized for fixed-size boxes.

Data defined on unions of rectangles of the form (5).

$$\phi : \Omega \to \mathcal{T}$$

where $\mathcal{T}$ is as in (**??**). These are represented using the class `Proto::LevelBoxData`, which implements a specialized version of the `Chombo::LevelData` API.
`LevelBoxData<T,NCOMPS,MEMTYPE,CENTERING>` is templated on the following:

- the value T (e.g. `float, double, int, long ...`) taken on by the constituent `BoxData`s;
- `MemType MEMTYPE` where `MemType` is an enumeration type which specifies whether the object is stored on host memory or device memory, i.e. `enum MemType{host,device}`;
- an unsigned int `NCOMPS`, which specifies the number of components (defaults to 1);
- an unsigned int `CENTERING`, where `CENTERING` encodes the centering of the data: `CENTERING` $= \boldsymbol{D}$ if the data is cell-centered (the default value); `CENTERING` $= d \in \{0, \ldots, \boldsymbol{D} - 1\}$ if the data is centered on the d-normal faces.

Unlike in `BoxData`, we only permit singly-indexed quantities as value types.

• Constructor.
`LevelBoxData(DisjointBoxLayout& a_dbl,int a_nGhost = 0)`,
`LevelBoxData::define(DisjointBoxLayout& a_dbl,int a_nGhost = 0)`.

For each Ω_k in the disjoint union, there is created on rank p_k a `BoxData` of the form

$$\phi^k : \Omega_k^{ghost} \to \mathcal{T} \ , \ \Omega_k^{ghost} = [\boldsymbol{i}_k\boldsymbol{r} - n_g\boldsymbol{u}, (\boldsymbol{i}_k + \boldsymbol{u})\boldsymbol{r} + (n_g - 1)\boldsymbol{u} + \boldsymbol{e}^d]$$

i.e. ϕ_k is a `BoxData` defined over $\mathcal{G}_{\boldsymbol{r}}(\Omega_k)$, plus an additional row of points on the high side in the d direction, for face-centered data in the d direction. We refer to $\Omega_k^{valid} =$

$[\boldsymbol{i}_k\boldsymbol{r}, (\boldsymbol{i}_k+\boldsymbol{u})\boldsymbol{r}-\boldsymbol{u}+\boldsymbol{e}^d]$ as the *valid region* and to the subset of the data defined on the valid region as the *valid data.* In the case of cell-centered `LevelBoxData`, there is exactly one valid region per point in Ω, so that the valid data in a `LevelBoxData` defines a single-valued map from Ω to $\mathcal{T}$. For face-centered data, there is overlap between adjacent boundaries at the low and high d-edges of the valid region. In structured-grid finite-volume methods, it is assumed that the computations filling the valid regions at the overlap are redundant and agree, at least to roundoff.

• Iterating through items in a `LevelBoxData`. The following loop iterates through all the `BoxData`s in a `LevelBoxData` on this MPI rank, using a standard C++ iterator syntax.

```
LevelBoxData<...> phiLevel(...);
for (auto dit = phiLevel.begin(); dit != phiLevel.end(); ++dit)
{
    BoxData<...>& phi = phiLevel[*dit];
    ...
}
```

`LevelBoxData::begin()` returns the same iterator as `DisjointBoxLayout::begin()`. Such an iterator can index into any `LevelBoxData` whose `DisJointBoxLayout` is related to the one defining the iterator by coarsening or refining.

• Moving data. There are three ways in which data can be moved to / from a `LevelBoxData`:

- Filling in ghost cell data. `void LevelBoxData::exchange()` copies data from all the valid regions in a `LevelBoxData` to the non-valid regions in the `BoxData` associated with each Ω_k, i.e.

$$\phi^k := \phi^{k'} \text{ on } \Omega_{k'}^{valid} \cap \Omega_k^{ghost} - \Omega_k^{valid} \ , \ k' \neq k.$$

- Copying from one `LevelBoxData` to another. These functions all have the form
  `LevelBoxData::copyTo(LevelBoxData<..>& a_dest ... )`,
  which copies the data containing in the source `*this` to the destination `a_dest`. For all versions of `copyTo`, the centerings of the source and destination must be the same. Given that is the case, then `copyTo` performs the operation

$$\phi^{dest,k_{dest}} := \phi^{src,k_{src}} \text{ on } \Omega_{k^{src}}^{valid} \cap \Omega_{k^{dest}}^{valid} \ , \text{ for all } k_{src}, k_{dest}.$$

- Copying between a `LevelBoxData` and a `Chombo::LevelData<FArrayBox>`, provided the `LevelBoxData` has `MemType = host`, the `LevelBoxData` is cell-centered, and the source and destination `DisjointBoxLayout`s are isomorphic. At the present time, number of components for the source and destination must also be the same, and all of the components are copied. These are expressed as member functions of
  `Chombo::LevelData<FArrayBox>`:
  `LevelData<FArrayBox>::copyTo(LevelBoxData<...>& a_dest, ...)`,
  `LevelData<FArrayBox>::copyFrom(LevelBoxData<...>& a_src, ...)`.